LEZIONE 12 - Ciclo for

PARTE TEORICA (2h)

1. Ciclo for: sintassi e semantica

Il ciclo `for` è il ciclo più utilizzato quando si conosce **in anticipo** il numero di iterazioni da eseguire. È particolarmente utile per contatori e iterazioni su sequenze.

Sintassi:

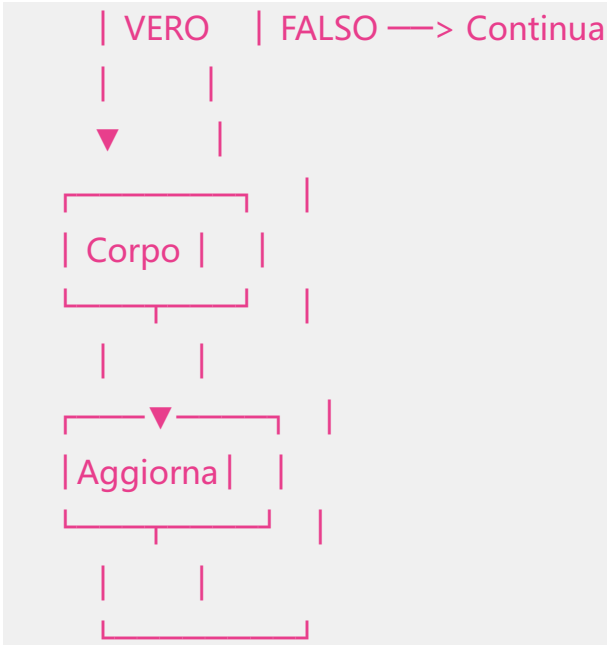
```
for (inizializzazione; condizione; aggiornamento) {
    // corpo del ciclo
}
```

Componenti del for:

1. **Inizializzazione**: eseguita una sola volta all'inizio (es: `i = 0`)
2. **Condizione**: verificata prima di ogni iterazione (es: `i < 10`)
3. **Aggiornamento**: eseguito dopo ogni iterazione (es: `i++`)
4. **Corpo**: blocco di istruzioni da ripetere

Flowchart del for:





Esempio base:

```c
#include <stdio.h>

int main() {
    // Stampa numeri da 1 a 10
    for (int i = 1; i <= 10; i++) {
        printf("%d ", i);
    }
    printf("\n");

    return 0;
}
// Output: 1 2 3 4 5 6 7 8 9 10
```

Equivalenza con while:

```c
// Ciclo for
for (int i = 0; i < 5; i++) {
    printf("%d\n", i);
}
```

```c
// Equivalente con while
int i = 0;          // inizializzazione
while (i < 5) {     // condizione
    printf("%d\n", i);
    i++;            // aggiornamento
}
```

---

2. Varianti del ciclo for

Contare all'indietro:

```c
for (int i = 10; i >= 1; i--) {
    printf("%d ", i);
}
printf("Partenza!\n");
// Output: 10 9 8 7 6 5 4 3 2 1 Partenza!
```

Incremento personalizzato:

```c
// Numeri pari da 0 a 20
for (int i = 0; i <= 20; i += 2) {
    printf("%d ", i);
}
// Output: 0 2 4 6 8 10 12 14 16 18 20

// Multipli di 5
for (int i = 5; i <= 50; i += 5) {
    printf("%d ", i);
}
// Output: 5 10 15 20 25 30 35 40 45 50
```

For con espressioni complesse:

```c
// Più variabili
for (int i = 0, j = 10; i < j; i++, j--) {
    printf("i=%d, j=%d\n", i, j);
}

// Condizione complessa
for (int i = 1; i <= 100 && i*i <= 50; i++) {
    printf("%d ", i);
}
```

For “infinito”:

```c
// Tutte le parti sono opzionali
for (;;) {
    printf("Ciclo infinito\n");
    // Serve un break per uscire!
}

// Equivalente a while(1)
```

---

3. Cicli annidati

I cicli annidati sono cicli dentro altri cicli. Il ciclo interno viene eseguito completamente per ogni iterazione del ciclo esterno.

Struttura base:

```c
for (int i = 0; i < righe; i++) {        // ciclo esterno
    for (int j = 0; j < colonne; j++) {  // ciclo interno
        // eseguito righe × colonne volte
    }
}
```

Esempio 1: Tabella di moltiplicazione

```c
#include <stdio.h>

int main() {
    printf("TABELLA PITAGORICA\n");
    printf("    ");

    // Intestazione
    for (int i = 1; i <= 10; i++) {
        printf("%4d", i);
    }
    printf("\n");
    printf("   ");
    for (int i = 0; i < 40; i++) printf("-");
    printf("\n");

    // Righe della tabella
    for (int i = 1; i <= 10; i++) {
        printf("%2d |", i);
        for (int j = 1; j <= 10; j++) {
            printf("%4d", i * j);
        }
        printf("\n");
    }

    return 0;
}
```

Esempio 2: Pattern con asterischi

```c
#include <stdio.h>

int main() {
```

```c
int n = 5;

// Triangolo rettangolo
printf("Triangolo:\n");
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        printf("* ");
    }
    printf("\n");
}

// Output:
// *
// * *
// * * *
// * * * *
// * * * * *

printf("\nQuadrato:\n");
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        printf("* ");
    }
    printf("\n");
}

// Output:
// * * * * *
// * * * * *
// * * * * *
// * * * * *
// * * * * *
```

```
    return 0;
}
```

Complessità dei cicli annidati:

```
// Un ciclo: O(n) - n iterazioni
for (int i = 0; i < n; i++) {
    // eseguito n volte
}

// Due cicli annidati: O(n²) - n×n iterazioni
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        // eseguito n×n volte
    }
}

// Tre cicli annidati: O(n³) - n×n×n iterazioni
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        for (int k = 0; k < n; k++) {
            // eseguito n×n×n volte
        }
    }
}
```

---

4. Break e Continue

Break: esce immediatamente dal ciclo

```
#include <stdio.h>

int main() {
```

```c
    // Cerca il primo numero divisibile per 7
    for (int i = 1; i <= 100; i++) {
        if (i % 7 == 0) {
            printf("Primo multiplo di 7: %d\n", i);
            break;  // Esce dal ciclo
        }
    }

    // Break in cicli annidati (esce solo dal ciclo interno)
    for (int i = 0; i < 3; i++) {
        printf("Ciclo esterno: %d\n", i);
        for (int j = 0; j < 5; j++) {
            if (j == 2) break;  // Esce solo dal for interno
            printf("  Ciclo interno: %d\n", j);
        }
    }

    return 0;
}
```

Continue: salta alla prossima iterazione

```c
#include <stdio.h>

int main() {
    // Stampa solo numeri dispari
    printf("Numeri dispari da 1 a 20:\n");
    for (int i = 1; i <= 20; i++) {
        if (i % 2 == 0) {
            continue;  // Salta i numeri pari
        }
        printf("%d ", i);
    }
    printf("\n");
```

```
    // Salta il numero 5
    printf("Numeri da 1 a 10 (escluso 5):\n");
    for (int i = 1; i <= 10; i++) {
        if (i == 5) continue;
        printf("%d ", i);
    }
    printf("\n");

    return 0;
}
```

Differenza tra break e continue:

```
// Con break
for (int i = 0; i < 10; i++) {
    if (i == 5) break;
    printf("%d ", i);
}
// Output: 0 1 2 3 4

// Con continue
for (int i = 0; i < 10; i++) {
    if (i == 5) continue;
    printf("%d ", i);
}
// Output: 0 1 2 3 4 6 7 8 9
```

---

5. Scelta del ciclo appropriato

Situazione	Ciclo consigliato	Esempio
Numero iterazioni noto	`for`	Stampare numeri da 1 a 100
Iterare su array/sequenza	`for`	Processare elementi array
Condizione di uscita variabile	`while`	Leggere input fino a valore speciale
Almeno un'esecuzione necessaria	`do-while`	Menu, validazione input
Cicli infiniti con break	`for(;;)` o `while(1)`	Server, game loop

Esempi comparativi:

```c
// FOR - quando sai quante volte
for (int i = 0; i < 10; i++) {
    printf("%d ", i);
}

// WHILE - quando la condizione è dinamica
int somma = 0, n;
while (somma < 100) {
    scanf("%d", &n);
    somma += n;
}

// DO-WHILE - quando serve almeno un'esecuzione
int scelta;
do {
    printf("Menu: ");
```

```
    scanf("%d", &scelta);
} while (scelta < 1 || scelta > 3);
```

---

6. Pattern comuni di iterazione

Sommatoria:

```
int somma = 0;
for (int i = 1; i <= n; i++) {
    somma += i;
}
```

Produttoria:

```
int prodotto = 1;
for (int i = 1; i <= n; i++) {
    prodotto *= i;
}
```

Ricerca:

```
int trovato = 0;
for (int i = 0; i < n; i++) {
    if (array[i] == target) {
        trovato = 1;
        break;
    }
}
```

Conteggio:

```
int contatore = 0;
for (int i = 0; i < n; i++) {
```

```
    if (condizione) {
        contatore++;
    }
}
```

PARTE PRATICA (2h)

Esercizio 1: Tabelle e sequenze numeriche

```c
#include <stdio.h>

int main() {
    int n;

    printf("Inserisci un numero: ");
    scanf("%d", &n);

    // Tabellina
    printf("\nTabellina del %d:\n", n);
    for (int i = 1; i <= 10; i++) {
        printf("%d x %d = %d\n", n, i, n * i);
    }

    // Quadrati
    printf("\nQuadrati da 1 a %d:\n", n);
    for (int i = 1; i <= n; i++) {
        printf("%d² = %d\n", i, i * i);
    }

    // Cubi
    printf("\nCubi da 1 a %d:\n", n);
    for (int i = 1; i <= n; i++) {
```

```
        printf("%d³ = %d\n", i, i * i * i);
    }

    return 0;
}
```

Esercizio 2: Disegno di pattern con asterischi

```
#include <stdio.h>

int main() {
    int n;

    printf("Inserisci l'altezza del pattern: ");
    scanf("%d", &n);

    // Pattern 1: Triangolo crescente
    printf("\nPattern 1 - Triangolo:\n");
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            printf("* ");
        }
        printf("\n");
    }

    // Pattern 2: Triangolo decrescente
    printf("\nPattern 2 - Triangolo invertito:\n");
    for (int i = n; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            printf("* ");
        }
        printf("\n");
    }
```

```
// Pattern 3: Piramide
printf("\nPattern 3 - Piramide:\n");
for (int i = 1; i <= n; i++) {
    // Spazi iniziali
    for (int j = 1; j <= n - i; j++) {
        printf("  ");
    }
    // Asterischi
    for (int j = 1; j <= 2 * i - 1; j++) {
        printf("* ");
    }
    printf("\n");
}

// Pattern 4: Rombo
printf("\nPattern 4 - Rombo:\n");
// Parte superiore
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n - i; j++) {
        printf(" ");
    }
    for (int j = 1; j <= 2 * i - 1; j++) {
        printf("*");
    }
    printf("\n");
}
// Parte inferiore
for (int i = n - 1; i >= 1; i--) {
    for (int j = 1; j <= n - i; j++) {
        printf(" ");
    }
    for (int j = 1; j <= 2 * i - 1; j++) {
        printf("*");
    }
```

```
        printf("\n");
    }

    return 0;
}
```

Esercizio 3: Sommatorie e produttorie

```
#include <stdio.h>

int main() {
    int n;

    printf("Inserisci un numero: ");
    scanf("%d", &n);

    // Somma dei primi n numeri naturali
    int somma = 0;
    for (int i = 1; i <= n; i++) {
        somma += i;
    }
    printf("\nSomma 1+2+...+%d = %d\n", n, somma);
    printf("Formula: %d*(%d+1)/2 = %d\n", n, n, n*(n+1)/2);

    // Somma dei quadrati
    int sommaQuadrati = 0;
    for (int i = 1; i <= n; i++) {
        sommaQuadrati += i * i;
    }
    printf("\nSomma 1²+2²+...+%d² = %d\n", n, sommaQuadrati);

    // Fattoriale
    long long fattoriale = 1;
```

```c
    for (int i = 1; i <= n; i++) {
        fattoriale *= i;
    }
    printf("\n%d! = %lld\n", n, fattoriale);

    // Potenza (n^k)
    int k;
    printf("\nInserisci l'esponente: ");
    scanf("%d", &k);

    long long potenza = 1;
    for (int i = 0; i < k; i++) {
        potenza *= n;
    }
    printf("%d^%d = %lld\n", n, k, potenza);

    return 0;
}
```

Esercizio 4: Ricerca di numeri primi

```c
#include <stdio.h>

int main() {
    int n;

    printf("Trova tutti i numeri primi fino a: ");
    scanf("%d", &n);

    printf("\nNumeri primi fino a %d:\n", n);

    int contatore = 0;
```

```c
    for (int num = 2; num <= n; num++) {
        int isPrimo = 1;  // Assumiamo che sia primo

        // Verifichiamo se num è divisibile per qualche numero
        for (int i = 2; i * i <= num; i++) {
            if (num % i == 0) {
                isPrimo = 0;  // Non è primo
                break;
            }
        }

        if (isPrimo) {
            printf("%d ", num);
            contatore++;
            if (contatore % 10 == 0) printf("\n");  // A capo ogni 10 numeri
        }
    }

    printf("\n\nTotale numeri primi: %d\n", contatore);

    // Verifica se un numero specifico è primo
    int test;
    printf("\nVerifica se un numero è primo: ");
    scanf("%d", &test);

    if (test < 2) {
        printf("%d non è primo\n", test);
    } else {
        int primo = 1;
        for (int i = 2; i * i <= test; i++) {
            if (test % i == 0) {
                primo = 0;
                printf("%d non è primo (divisibile per %d)\n", test, i);
                break;
            }
```

```
        }
        if (primo) {
            printf("%d è primo!\n", test);
        }
    }

    return 0;
}
```

Esercizio 5: Tavola degli scacchi

```
#include <stdio.h>

int main() {
    int dimensione = 8;

    printf("TAVOLA DEGLI SCACCHI (%dx%d)\n\n", dimensione, dimensione);

    // Metodo 1: con caratteri
    for (int i = 0; i < dimensione; i++) {
        for (int j = 0; j < dimensione; j++) {
            if ((i + j) % 2 == 0) {
                printf("■ ");  // Casella scura
            } else {
                printf("  ");  // Casella chiara
            }
        }
        printf("\n");
    }

    printf("\n");

    // Metodo 2: con numeri
```

```
    for (int i = 0; i < dimensione; i++) {
        for (int j = 0; j < dimensione; j++) {
            if ((i + j) % 2 == 0) {
                printf("0 ");
            } else {
                printf("1 ");
            }
        }
        printf("\n");
    }

    return 0;
}
```

Esercizi da svolgere autonomamente:

1. **Calendario**: Crea un programma che stampa i giorni di un mese (da 1 a 30/31) disposti in righe da 7 giorni
2. **Serie di Fibonacci**: Stampa i primi N numeri della serie di Fibonacci usando un ciclo for
3. **Triangolo di Floyd**: Stampa il triangolo di Floyd (numeri consecutivi disposti a triangolo)

   ```
   1
   2 3
   4 5 6
   7 8 9 10
   ```

4. **Statistica numeri**: Chiedi N numeri all'utente e calcola: somma, media, massimo, minimo, quanti sono pari/dispari
5. **Conversione binario**: Converti un numero decimale in binario usando il ciclo for e l'operatore modulo

6. **Triangolo di Pascal**: Stampa le prime N righe del triangolo di Pascal
7. **MCD ed mcm**: Calcola il Massimo Comun Divisore e il minimo comune multiplo di due numeri
8. **Numeri perfetti**: Trova tutti i numeri perfetti fino a N (un numero è perfetto se è uguale alla somma dei suoi divisori propri, es: 6 = 1+2+3)
9. **Tabella ASCII**: Stampa la tabella ASCII dei caratteri stampabili (da 32 a 126)
10. **Stella a 5 punte**: Disegna una stella a 5 punte usando cicli annidati e asterischi

---

Riepilogo Concetti Chiave

✓ Il ciclo **for** è ideale quando si conosce il numero di iterazioni
✓ Struttura: `for (init; condizione; aggiornamento) { corpo }`
✓ I **cicli annidati** moltiplicano il numero di iterazioni
✓ **Break** esce dal ciclo, **continue** salta all'iterazione successiva
✓ Scegli il ciclo in base al problema: for, while o do-while
✓ Pattern comuni: sommatoria, produttoria, ricerca, conteggio

Prossima lezione: Approfondiremo gli **array monodimensionali**!